

QUIC vs UDP

First Byte

Long Header: MSB = 1 ($\geq 0x80$)

Short Header: MSB = 0

Fixed Bit

2nd MSB = 1 for QUIC Packets (Long Header: 0xC0 to 0xFF, Short Header: 0x40 to 0x70)

Version Field (4 bytes after the first bytes)

Standard QUIC: 0x00000001

Connection ID

Look like random noise, but their positional consistency across multiple packets is a dead giveaway (compared to unstructured payload of a generic UDP stream).

Initial Packets

Contains the unencrypted handshakes (though integrity-protected)

CRYPTO FRAME

Wraps the TLS 1.3 ClientHello. Even though initial payload is encrypted, policers can use publicly known salt to derive the keys. So, any policer can decrypt the initial packet's payload.

What we can extract:

- SNI (Server Name Identification): Hostname trying to be reached.
- ALPN (Application-Layer Protocol Negotiation): Confirms the protocol, usually **h3** for **HTTPS/3**.
- Fingerprint (JA4Q/Cynefin): You can hash specific set of extensions, cipher suites, and versions the client supports. This allows to identify the **browser or app**.

ENCRYPTED DETAILS WE CAN GET FROM QUIC HEADERS

The Identity

For the **API Classification** (Google/Facebook/Youtube), we need the **Server Name Indication (SNI)**.

SNI can be found inside the 'Client Hello' in the CRYPTO frame encrypted using 'Initial Keys'.

```
TLShv1.3 Record Layer: Handshake Protocol: Client Hello
  Handshake Protocol: Client Hello
    Handshake Type: Client Hello (1)
    Length: 1728
    > Version: TLS 1.2 (0x0303)
    Random: 79cefffc2aecac3f2e83c65f97de2b133d517dac259d20ce87
    Session ID Length: 0
    Cipher Suites Length: 6
    > Cipher Suites (3 suites)
    Compression Methods Length: 1
    > Compression Methods (1 method)
    Extensions Length: 1681
    > Extension: server_name (len=20) name=www.youtube.com
      Type: server_name (0)
      Length: 20
      > Server Name Indication extension
        Server Name list length: 18
        Server Name Type: host_name (0)
        Server Name length: 15
        Server Name: www.youtube.com
```

Classification Layer (What are they doing)

Feature	QUIC Web (Browsing)	QUIC Streaming (Video)	QUIC API (Mobile/Apps)
Flow Duration	Short (seconds)	Long (minutes)	Short/Burst (milliseconds)
Packet Sizes	High Variance (Mix of small/large)	Consistently Large (~1350B)	Mostly Small/Medium
Stream IDs	Many IDs (0, 4, 8, 12...)	Few IDs (usually just one for video)	Very few IDs
Burstiness	High (loading a page)	Periodic (buffering chunks)	Request-Response pairs

Type Classification

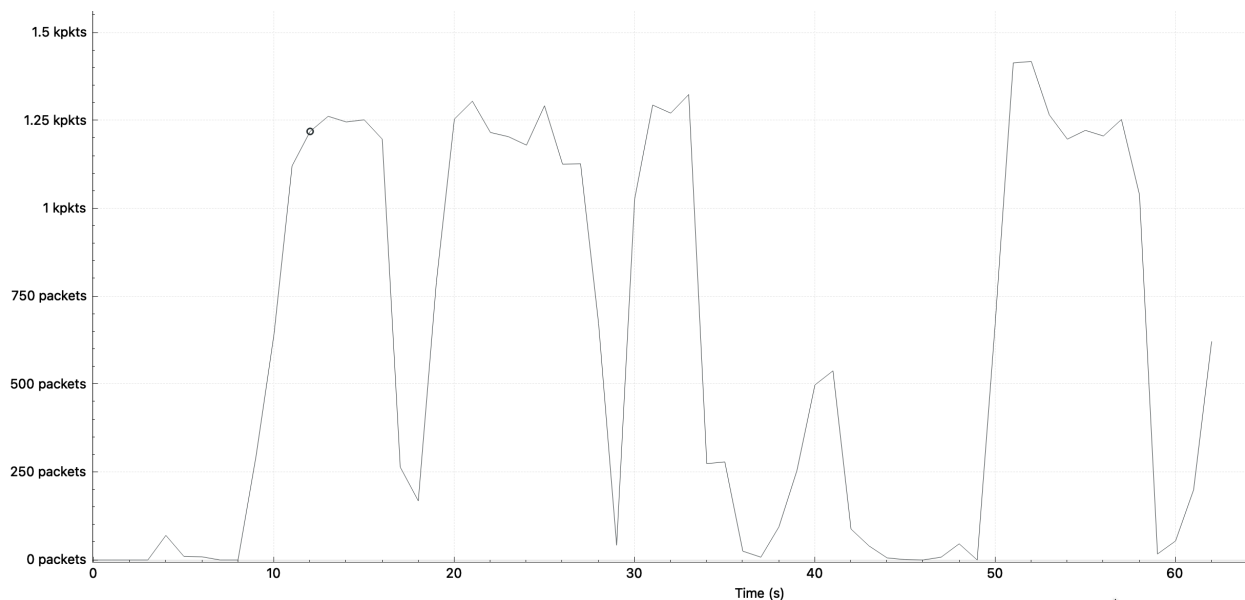
We need to look at the SST: Shape, Size & Time

We can use **Traffic Fingerprinting** based on the behavior of the packets

Patterns

Video

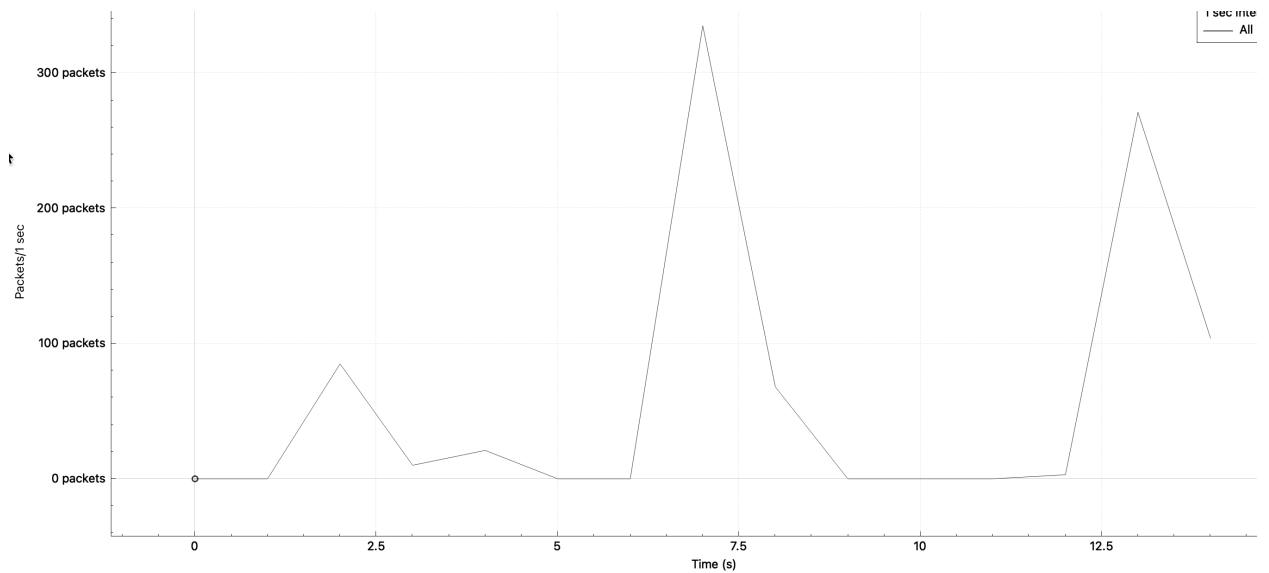
- Packet Size: Large, Consistently MTU-sized (~1200+ bytes)
- Directionality: Heavy Downlink (Server -> Client)
- Inter-Arrival Time: Steady, high-frequency stream



We can see massive bursts of 1350 byte packets (filling the buffer) followed by 2-5 seconds of near-total silence.

Web

- Packet Size: Bursty, mixed large and small packets
- Directionality: Interactive; back-and-forth spikes
- Inter-Arrival Time: Irregular; rapid bursts followed by silence



Irregular Timing Pattern. Spikes only happen when we click or scroll. No rhythmic “sawtooth” like in Video.

Chat/VoIP

- Packet Size: Small, Frequent Packets (~100-400 bytes)
- Directionality: Symmetric (roughly equal both ways)
- Inter-Arrival Time: Constant, low-latency intervals

While we can't see the Stream IDs in a protected packet, we can see the multiplexing through **sudden changes in IAT**. If a steady stream of video suddenly has tiny packets interleaved within it, your classifier can flag that a "secondary stream" (like a chat message or an ad-tracking pixel) has just been triggered.